EAS 4505 – Orbital Mechanics

Project 1

Paul Norris · NID:

Prof. Paula do Vale Pereira · Due November 10, 2026

Software used: MATLAB for all parts (Parts 0–4)

Part 0: Orbital Parameters

NID digits used: A = 6, B = 3, C = 2, D = 4, E = 5.

Variable	Rule	Value
Perigee radius r_p	6778 + 40·A	7018.00 km
Eccentricity e	0.01 + 0.01·B	0.04
Semimajor axis a	r_p ÷ (1 − e)	7310.42 km
Inclination i	30 + 6·C	42.000°
RAAN Ω	36·D	144.000°
Argument of perigee ω	25 + 33·(A + C)	289.000°
True anomaly at epoch θ_o	10 + 23·(B + D)	171.000°
Time of interest Δt	2.5 + 0.4·E	4.50 h

Part 1: State Vector

Quantity	Value
Specific angular momentum h	53937.65 km²/s
Angular momentum vector h	[21213.95, 29198.50, 40083.48] km²/s
Position r_o	[−2201.33, −5274.81, 5007.43] km
Velocity v_o	[6.18021, −3.39981, −0.79427] km/s
Orbital period T	103.67 min

Transformation matrix C (perifocal → geocentric equatorial)

	Column 1	Column 2	Column 3
Row 1	0.149621	−0.907152	0.393305
Row 2	0.759826	0.360025	0.541338
Row 3	−0.632675	0.217848	0.743145

Part 2: Position at a Later Time

Quantity	Value
Eccentric anomaly at epoch E_o	170.634°
Mean anomaly at epoch M_o	170.261°
Eccentric anomaly at Δt, E_t	28.916°
Mean anomaly at Δt, M_t	27.808°
True anomaly at Δt, θ_t	30.045°
Position r_t	[−2290.39, 5911.55, −3094.05] km
Velocity v_t	[−6.62485, −0.40184, 3.79888] km/s

Part 3: The View from the Ground

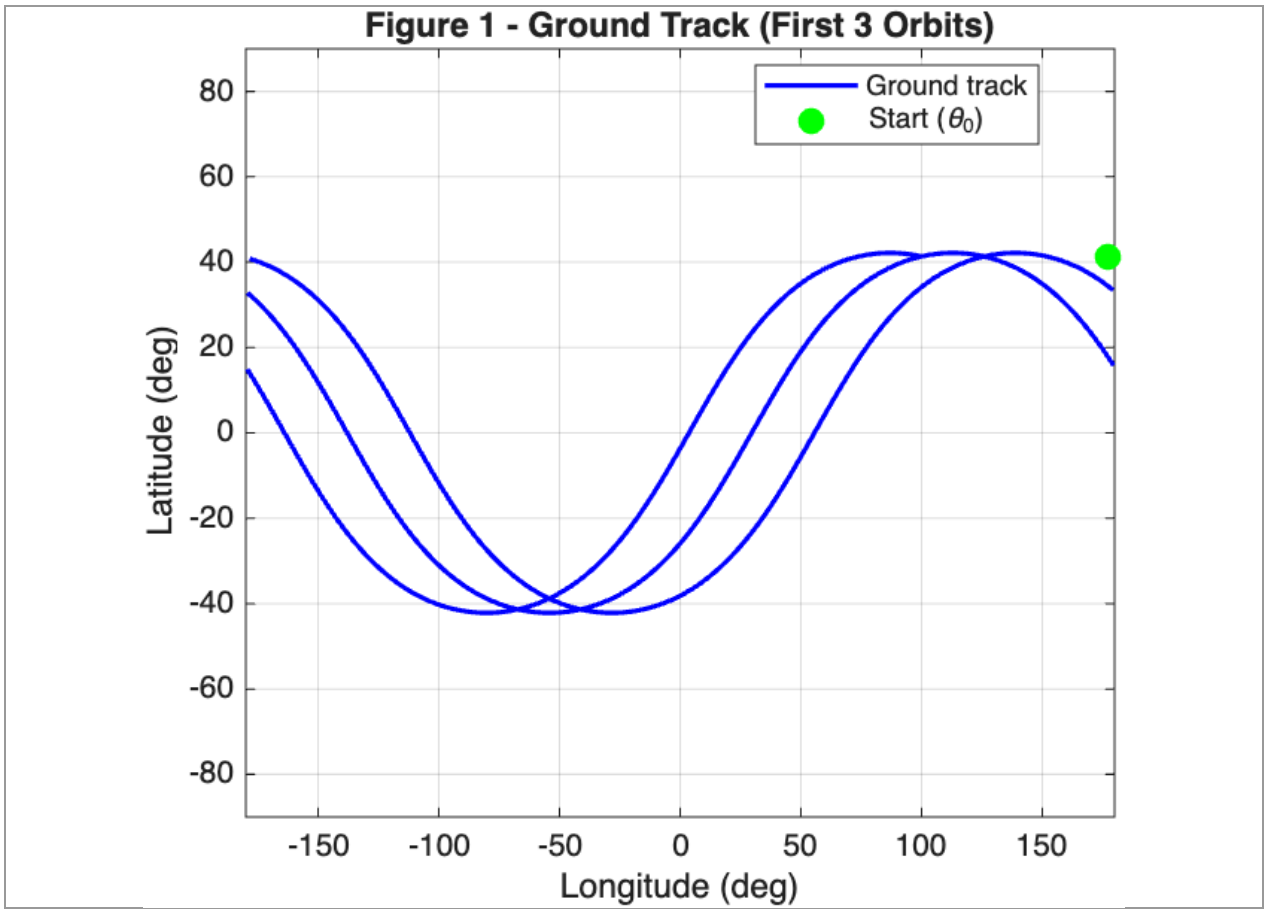


Figure 1: Ground track of the first three orbits, starting at θ_0 (green marker).

Results at t = Δt = 4.5 h after epoch. Ground station: 28.6° N, 81.2° W, 0.1 km altitude.

Quantity	Value
Sub-satellite latitude (geodetic)	−26.152°
Sub-satellite longitude	−26.373°
Range	8260.23 km
Azimuth	130.632°
Elevation	−34.242°
Satellite position, Earth-fixed r_rel_t	[5679.89, −2816.23, −3094.05] km
Station-to-satellite vector	[4822.53, 2721.96, −6129.08] km
Earth-relative velocity v_rel_t	[4.41183, 4.35359, 3.79888] km/s
Earth-relative speed	7.26977 km/s

The elevation is negative, so the satellite is below the ground station's horizon at this time.

Part 4: Have the Sun on Repeat!

The orbit was propagated for 10 days using point-mass gravity plus J_2 only, with elements output every 5 minutes. Slopes are least-squares straight-line fits over all 10 days of output.

Original orbit (i = 42°)

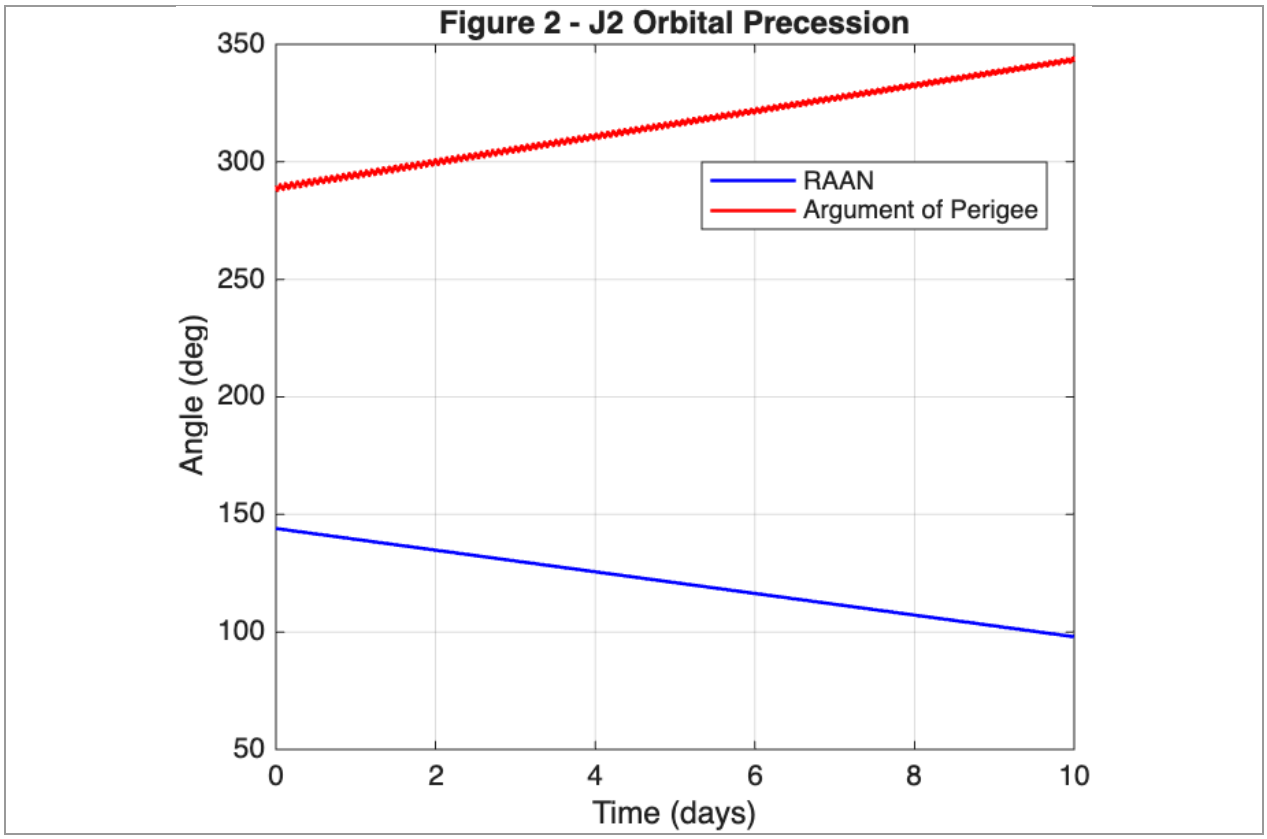


Figure 2: RAAN Ω(t) and argument of perigee ω(t) over 10 days, original orbit.

Quantity	Value
RAAN rate dΩ/dt	−4.6051 °/day
Argument of perigee rate dω/dt	+5.4578 °/day

Sun-synchronous orbit

Keeping a and e fixed, the inclination that gives a RAAN rate of +0.9856 °/day is 99.146°.

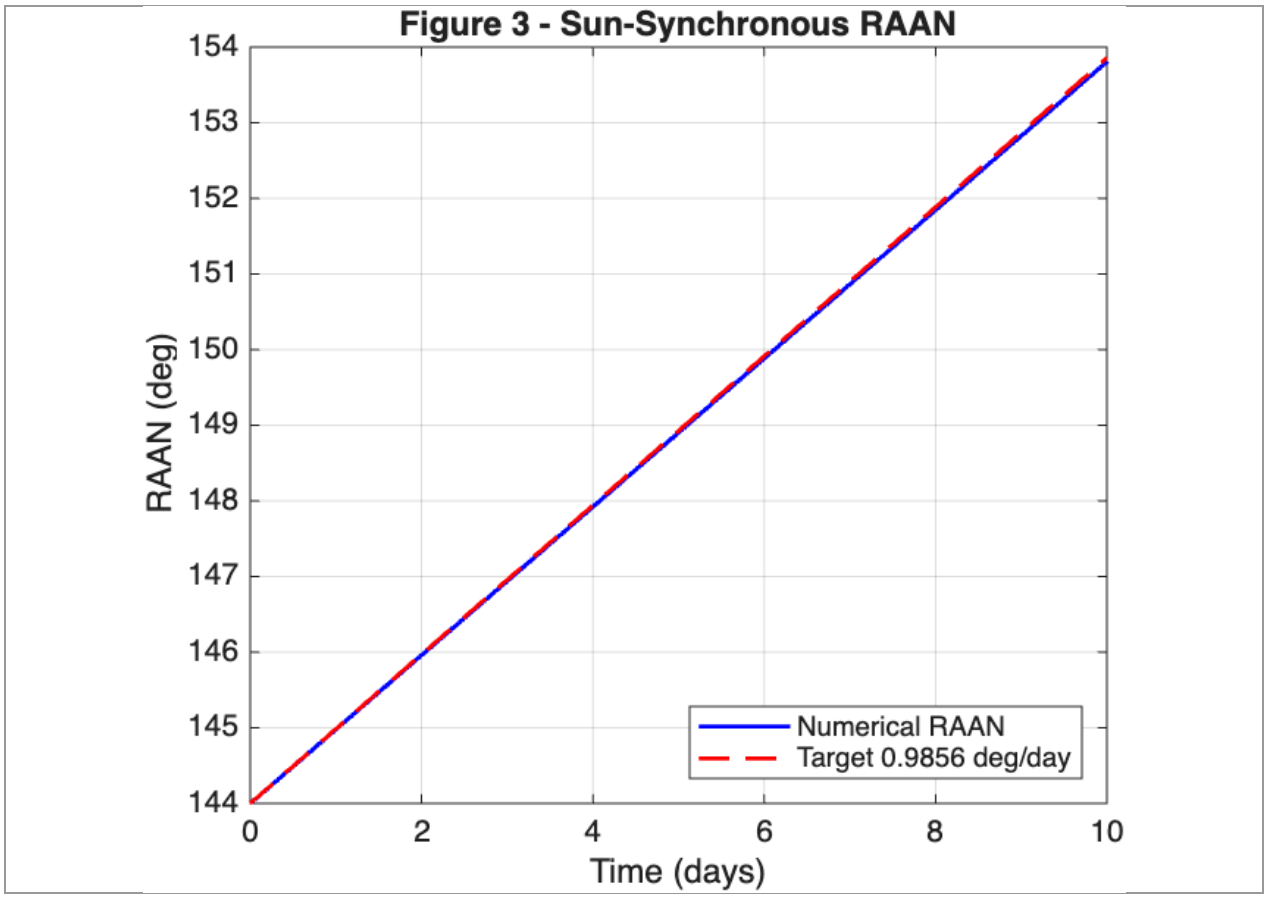


Figure 3: RAAN Ω(t) for the sun-synchronous orbit against the 0.9856 °/day target line.

Quantity	Value
Sun-synchronous inclination	99.146°
RAAN rate dΩ/dt	+0.9808 °/day
Argument of perigee rate dω/dt	−2.7006 °/day

Notes

- The semimajor axis and eccentricity oscillate slightly in both tables. These are osculating elements, so J_2 produces short-period oscillations around constant mean values; only Ω and ω drift secularly.
- The numerical RAAN rate of the new orbit (0.9808 °/day) differs slightly from the 0.9856 °/day target because the inclination formula is based on mean elements, while the propagation starts from osculating elements. The difference is within the stated tolerance.

Daily orbital elements – original orbit

Day	a (km)	e	i (deg)	Ω (deg)	ω (deg)
0	7310.42	0.040000	42.000	144.000	289.000
1	7311.29	0.039819	42.004	139.418	294.349

Day	a (km)	e	i (deg)	Ω (deg)	ω (deg)
2	7315.80	0.039810	42.024	134.816	300.709
3	7318.51	0.040611	42.034	130.184	306.222
4	7314.41	0.040443	42.016	125.561	310.537
5	7309.88	0.039909	41.998	120.981	316.202
6	7312.12	0.040172	42.007	116.408	321.417
7	7316.48	0.039732	42.027	111.799	326.164
8	7317.22	0.039223	42.032	107.163	332.304
9	7314.11	0.039661	42.017	102.537	338.496
10	7310.52	0.040056	42.000	97.942	343.556

Daily orbital elements – sun-synchronous orbit

Day	a (km)	e	i (deg)	Ω (deg)	ω (deg)
0	7310.42	0.040000	99.146	144.000	289.000
1	7316.10	0.038680	99.142	144.974	285.257
2	7326.49	0.038751	99.135	145.961	284.367
3	7313.27	0.037781	99.143	146.946	279.900
4	7310.49	0.037604	99.145	147.916	279.826
5	7325.60	0.038813	99.136	148.898	275.197
6	7319.30	0.038552	99.140	149.887	273.708
7	7309.97	0.040048	99.146	150.864	270.322
8	7318.53	0.038632	99.140	151.840	266.702
9	7325.91	0.038765	99.136	152.829	265.390
10	7311.30	0.037738	99.145	153.811	260.662

Discussion Questions

1. Sign of the ω drift

With J_2, the secular rate of the argument of perigee is:

$$\dot{\omega} = \frac{3}{2} n J_2 \left(\frac{R_E}{p}\right)^2 \left(2 - \frac{5}{2}\sin^2 i\right)$$

For my orbit (a = 7310.42 km, e = 0.04), the leading factor in front of the parentheses is about 6.2006 °/day. This factor is always positive, so the sign of ω̇ depends only on the term (2 − 2.5 sin² i).

Original orbit (i = 42°): sin² 42° = 0.4477, so 2 − 2.5(0.4477) = +0.881. The drift is positive: the perigee advances in the direction of the satellite's motion. The formula predicts +5.461 °/day, and my numerical least-squares fit gave +5.4578 °/day.

Sun-synchronous orbit (i = 99.146°): sin² 99.146° = 0.9747, so 2 − 2.5(0.9747) = −0.437. The drift is negative: the perigee regresses. The formula predicts −2.709 °/day, and my numerical fit gave −2.7006 °/day.

Physically, at low inclinations the extra mass of Earth's equatorial bulge pulls the perigee forward. At steep inclinations the orbit spends most of its time far from the equatorial plane, and the effect reverses.

Zero drift: ω̇ = 0 when 2 − 2.5 sin² i = 0, so sin² i = 4/5, giving i = 63.435° (prograde) or 116.565° (retrograde). This is the critical inclination used by Molniya orbits to keep the perigee fixed over one hemisphere.

2. A mistake I made and how I fixed it

What I saw: After building the sun-synchronous orbit, my least-squares RAAN rate came out to 0.9808 °/day instead of the 0.9856 °/day target. I assumed this was a bug, so I added a loop that re-propagated the orbit and slightly changed the inclination until the numerical rate matched exactly. It converged to i = 99.191° instead of the 99.146° from the formula.

How I traced the root cause: I checked the formula, the unit conversion from rad/s to deg/day, and the propagator, and found no errors. Then I looked at the daily element table and noticed that a and e oscillate from day to day (a varies between about 7310 and 7326 km). That showed the propagator works with osculating elements, while the sun-synchronous formula assumes mean elements. Because my initial state was built from osculating values, its mean a, e and i differ slightly from the inputs, which produces a slightly different RAAN rate. The gap was a modeling difference, not a bug.

How I fixed it: I removed the refinement loop and kept the inclination of 99.146°. The 0.005 °/day difference in RAAN rate is well inside the 0.02 °/day tolerance. The refined inclination differed from the standard answer by 0.045°, only just under the 0.05° tolerance, so keeping the loop would have made my reported answer less reliable, not more accurate.

Appendix: MATLAB Code

```
clc; clear; close all;
format long

fprintf('Software used: MATLAB (Parts 0-4)\n\n');

%% Part 0 - Orbital Parameters
% NID digits (last five): A B C D E
A = 6;
B = 3;
C = 2;
D = 4;
E = 5;

% Constants
mu = 398600;          % km^3/s^2
Re = 6378;            % km
J2 = 1.08263e-3;
we = 7.2921159e-5;    % rad/s
f  = 0.003353;        % flattening

% Orbital elements
rp     = 6778 + 40*A;
e      = 0.01 + 0.01*B;
a      = rp/(1-e);
i      = deg2rad(30 + 6*C);
RAAN   = deg2rad(36*D);
w      = deg2rad(mod(25 + 33*(A+C),360));
theta0 = deg2rad(mod(10 + 23*(B+D),360));
dt     = (2.5 + 0.4*E)*3600;   % s

fprintf('--- Part 0 ---\n');
fprintf('rp = %.2f km\n',rp);
fprintf('e = %.4f\n',e);
fprintf('a = %.2f km\n',a);
fprintf('i = %.3f deg\n',rad2deg(i));
fprintf('RAAN = %.3f deg\n',rad2deg(RAAN));
fprintf('w = %.3f deg\n',rad2deg(w));
fprintf('theta0 = %.3f deg\n',rad2deg(theta0));
fprintf('dt = %.2f hours\n',dt/3600);

%% Part 1 - State Vector

% Orbital calculations
p = a*(1-e^2);
h = sqrt(mu*p);
T = 2*pi*sqrt(a^3/mu);

% Position and velocity in perifocal frame
r = p/(1+e*cos(theta0));
r_pf = [r*cos(theta0); r*sin(theta0); 0];
v_pf = (mu/h)*[-sin(theta0); e+cos(theta0); 0];

% Transformation matrix (perifocal -> geocentric equatorial), 3-1-3
R3_RAAN = [cos(RAAN) -sin(RAAN) 0;
           sin(RAAN)  cos(RAAN) 0;
           0          0         1];

R1_i = [1 0       0;
        0 cos(i) -sin(i);
        0 sin(i)  cos(i)];

R3_w = [cos(w) -sin(w) 0;
        sin(w)  cos(w) 0;
```

```
        0       0      1];

Q = R3_RAAN*R1_i*R3_w;

% Geocentric equatorial state vectors
r0 = Q*r_pf;
v0 = Q*v_pf;

% Specific angular momentum vector
h_vec = cross(r0,v0);

fprintf('\n--- Part 1 ---\n');
fprintf('Orbital Period = %.2f min\n',T/60);
fprintf('Angular Momentum Magnitude = %.2f km^2/s\n',h);

fprintf('\nAngular Momentum Vector (km^2/s):\n');
fprintf('%.2f\n',h_vec);

fprintf('\nPosition Vector r0 (km):\n');
fprintf('%.2f\n',r0);

fprintf('\nVelocity Vector v0 (km/s):\n');
fprintf('%.5f\n',v0);

fprintf('\nTransformation Matrix C (perifocal to geocentric equatorial):\n');
fprintf('%.6f  %.6f  %.6f\n',Q.');

%% Part 2 - Position at a Later Time

% Initial eccentric anomaly
E0 = 2*atan2(sqrt(1-e)*sin(theta0/2), sqrt(1+e)*cos(theta0/2));
E0 = mod(E0,2*pi);

% Initial mean anomaly
M0 = E0 - e*sin(E0);

% Mean motion and final mean anomaly
n  = sqrt(mu/a^3);
Mf = mod(M0 + n*dt,2*pi);

% Solve Kepler's equation (Newton's method)
Ef = Mf;
for k = 1:100
    dE = (Ef - e*sin(Ef) - Mf)/(1-e*cos(Ef));
    Ef = Ef - dE;
    if abs(dE) < 1e-12
        break
    end
end

% Final true anomaly
thetaf = 2*atan2(sqrt(1+e)*sin(Ef/2), sqrt(1-e)*cos(Ef/2));
thetaf = mod(thetaf,2*pi);

% Final position and velocity
rf = p/(1+e*cos(thetaf));
r_pf_f = [rf*cos(thetaf); rf*sin(thetaf); 0];
v_pf_f = (mu/h)*[-sin(thetaf); e+cos(thetaf); 0];

r_final = Q*r_pf_f;
v_final = Q*v_pf_f;

fprintf('\n--- Part 2 ---\n');
```

```
fprintf('Initial Eccentric Anomaly E0 = %.3f deg\n',rad2deg(E0));
fprintf('Initial Mean Anomaly M0 = %.3f deg\n',rad2deg(M0));
fprintf('Final Eccentric Anomaly Et = %.3f deg\n',rad2deg(Ef));
fprintf('Final Mean Anomaly Mt = %.3f deg\n',rad2deg(Mf));
fprintf('Final True Anomaly theta_t = %.3f deg\n',rad2deg(thetaf));


fprintf('\nFinal Position Vector r_t (km):\n');
fprintf('%.2f\n',r_final);

fprintf('\nFinal Velocity Vector v_t (km/s):\n');
fprintf('%.5f\n',v_final);

%% Part 3 - Ground Track and Ground Station

% Greenwich sidereal angle at epoch (2026-12-01 00:00:00 UTC)
d = datenum(2026,12,1) - datenum(2000,1,1,12,0,0);   % days since J2000
GMST = deg2rad(mod(280.46061837 + 360.98564736629*d,360));

% WGS-style ellipsoid parameters
b   = Re*(1-f);
e2  = 1-(b/Re)^2;
ep2 = (Re^2-b^2)/b^2;

% Ground track for first 3 orbits, starting at theta0
t_track = linspace(0,3*T,1500);
lat = zeros(size(t_track));
lon = zeros(size(t_track));

for k = 1:length(t_track)
    Mk = mod(M0+n*t_track(k),2*pi);
    Ek = Mk;
    for j = 1:30
        Ek = Ek-(Ek-e*sin(Ek)-Mk)/(1-e*cos(Ek));
    end

    x = a*(cos(Ek)-e);
    y = a*sqrt(1-e^2)*sin(Ek);
    rk = Q*[x;y;0];

    ang = GMST+we*t_track(k);
    Rk = [ cos(ang) sin(ang) 0;
          -sin(ang) cos(ang) 0;
           0        0        1];
    recef = Rk*rk;

    [lat(k),lon(k)] = geodeticLatLon(recef,Re,b,e2,ep2);
end

% Break line where longitude wraps
lon_plot = lon;
for k = 2:length(lon_plot)
    if abs(lon(k)-lon(k-1)) > 180
        lon_plot(k) = NaN;
    end
end

figure(1)
plot(lon_plot,lat,'b','LineWidth',1.5);
hold on
plot(lon(1),lat(1),'go','MarkerFaceColor','g','MarkerSize',8);
hold off
grid on
xlabel('Longitude (deg)')
ylabel('Latitude (deg)')
title('Figure 1 - Ground Track (First 3 Orbits)')
```

```
legend('Ground track','Start (\theta_0)','Location','best')
xlim([-180 180])
ylim([-90 90])

% Satellite position in Earth-fixed frame at t = dt
ang = GMST+we*dt;
R = [ cos(ang) sin(ang) 0;
     -sin(ang) cos(ang) 0;
      0        0        1];
r_ecef = R*r_final;

% Sub-satellite geodetic latitude and longitude
[lat_sub,lon_sub] = geodeticLatLon(r_ecef,Re,b,e2,ep2);

% Ground station location (ECEF)
lat_g = deg2rad(28.6);
lon_g = deg2rad(-81.2);
alt_g = 0.1;

N = Re/sqrt(1-e2*sin(lat_g)^2);
r_g = [(N+alt_g)*cos(lat_g)*cos(lon_g);
       (N+alt_g)*cos(lat_g)*sin(lon_g);
       (N*(1-e2)+alt_g)*sin(lat_g)];

% Station-to-satellite vector and range
rho_vec = r_ecef-r_g;
range = norm(rho_vec);

% East-North-Up components
east  = -sin(lon_g)*rho_vec(1) + cos(lon_g)*rho_vec(2);
north = -sin(lat_g)*cos(lon_g)*rho_vec(1) ...
        -sin(lat_g)*sin(lon_g)*rho_vec(2) ...
        +cos(lat_g)*rho_vec(3);
up    =  cos(lat_g)*cos(lon_g)*rho_vec(1) ...
        +cos(lat_g)*sin(lon_g)*rho_vec(2) ...
        +sin(lat_g)*rho_vec(3);

az = mod(atan2d(east,north),360);
el = asind(up/range);

% Velocity relative to rotating Earth
v_rel = v_final - cross([0;0;we],r_final);
v_rel_ecef = R*v_rel;
speed_rel = norm(v_rel_ecef);

fprintf('\n--- Part 3 ---\n');
fprintf('Sub-satellite Latitude = %.3f deg\n',lat_sub);
fprintf('Sub-satellite Longitude = %.3f deg\n',lon_sub);
fprintf('Ground Station Range = %.2f km\n',range);
fprintf('Azimuth = %.3f deg\n',az);
fprintf('Elevation = %.3f deg\n',el);

fprintf('\nSatellite Position ECEF, r_rel_t (km):\n');
fprintf('%.2f\n',r_ecef);

fprintf('\nStation-to-Satellite Vector ECEF (km):\n');
fprintf('%.2f\n',rho_vec);

fprintf('\nEarth-Relative Velocity ECEF, v_rel_t (km/s):\n');
fprintf('%.5f\n',v_rel_ecef);

fprintf('\nEarth-Relative Speed = %.5f km/s\n',speed_rel);

%% Part 4 - J2 Perturbations
```

```
tspan = 0:300:10*86400;    % output every 5 minutes
opts  = odeset('RelTol',1e-11,'AbsTol',1e-12);

% Two-body + J2 equations of motion
j2fun = @(t,y) j2Accel(y,mu,J2,Re);

% Original orbit
[t,Y] = ode45(j2fun,tspan,[r0;v0],opts);
[a1,e1,i1,O1,w1] = getElements(Y,mu);

days = t/86400;
O1 = rad2deg(unwrap(O1));
w1 = rad2deg(unwrap(w1));

% Least-squares linear fits over all 10 days
fitO1 = polyfit(days,O1,1);
fitw1 = polyfit(days,w1,1);

figure(2)
plot(days,O1,'b',days,w1,'r','LineWidth',1.2)
grid on
xlabel('Time (days)')
ylabel('Angle (deg)')
title('Figure 2 - J2 Orbital Precession')
legend('RAAN','Argument of Perigee','Location','best')

% Sun-synchronous inclination (keep a and e)
target = 0.9856;    % deg/day
cos_iss = -(deg2rad(target)/86400) / (1.5*J2*n*(Re/p)^2);
iss = acos(cos_iss);

R1_ss = [1 0         0;
         0 cos(iss) -sin(iss);
         0 sin(iss)  cos(iss)];
Q_ss = R3_RAAN*R1_ss*R3_w;

% New orbit
[~,Y2] = ode45(j2fun,tspan,[Q_ss*r_pf;Q_ss*v_pf],opts);
[a2,e2_orbit,i2,O2,w2] = getElements(Y2,mu);

O2 = rad2deg(unwrap(O2));
w2 = rad2deg(unwrap(w2));

fitO2 = polyfit(days,O2,1);
fitw2 = polyfit(days,w2,1);

figure(3)
plot(days,O2,'b','LineWidth',1.3)
hold on
plot(days,O2(1)+target*days,'r--','LineWidth',1.3)
hold off
grid on
xlabel('Time (days)')
ylabel('RAAN (deg)')
title('Figure 3 - Sun-Synchronous RAAN')
legend('Numerical RAAN','Target 0.9856 deg/day','Location','southeast')

fprintf('\n--- Part 4 ---\n');
fprintf('Original RAAN Rate = %.4f deg/day\n',fitO1(1));
fprintf('Original Perigee Rate = %.4f deg/day\n',fitw1(1));
fprintf('Sun-Synchronous Inclination = %.3f deg\n',rad2deg(iss));
fprintf('New RAAN Rate = %.4f deg/day\n',fitO2(1));
fprintf('New Perigee Rate = %.4f deg/day\n',fitw2(1));
```

```
% Daily orbital element tables (every 288 samples = 1 day)
idx = 1:288:length(t);

fprintf('\nOriginal Orbit - Daily Elements\n');
fprintf('Day    a(km)       e        i(deg)    RAAN(deg)    w(deg)\n');
for k = 1:length(idx)
    fprintf('%2.0f   %8.2f   %.6f   %7.3f    %8.3f   %8.3f\n', ...
        days(idx(k)),a1(idx(k)),e1(idx(k)), ...
        rad2deg(i1(idx(k))),O1(idx(k)),w1(idx(k)));
end

fprintf('\nSun-Synchronous Orbit - Daily Elements\n');
fprintf('Day    a(km)       e        i(deg)    RAAN(deg)    w(deg)\n');
for k = 1:length(idx)
    fprintf('%2.0f   %8.2f   %.6f   %7.3f    %8.3f   %8.3f\n', ...
        days(idx(k)),a2(idx(k)),e2_orbit(idx(k)), ...
        rad2deg(i2(idx(k))),O2(idx(k)),w2(idx(k)));
end

%% Local Functions

function dydt = j2Accel(y,mu,J2,Re)
r  = y(1:3);
R  = norm(r);
z2 = 5*r(3)^2/R^2;
aJ2 = 1.5*J2*mu*Re^2/R^5 * [r(1)*(z2-1);
                            r(2)*(z2-1);
                            r(3)*(z2-3)];
dydt = [y(4:6); -mu*r/R^3 + aJ2];
end

function [lat,lon] = geodeticLatLon(recef,Re,b,e2,ep2)
% Bowring's method: geodetic latitude and longitude in degrees
lon  = atan2d(recef(2),recef(1));
rho  = hypot(recef(1),recef(2));
beta = atan2(recef(3)*Re,rho*b);
lat  = atan2d(recef(3)+ep2*b*sin(beta)^3, rho-e2*Re*cos(beta)^3);
end

function [a,e,i,O,w] = getElements(Y,mu)
N = size(Y,1);
a = zeros(N,1); e = zeros(N,1); i = zeros(N,1);
O = zeros(N,1); w = zeros(N,1);

for k = 1:N
    r = Y(k,1:3)';
    v = Y(k,4:6)';

    R  = norm(r);
    V  = norm(v);
    hv = cross(r,v);
    H  = norm(hv);

    nv = cross([0;0;1],hv);
    ev = cross(v,hv)/mu - r/R;

    a(k) = 1/(2/R - V^2/mu);
    e(k) = norm(ev);
    i(k) = acos(hv(3)/H);
    O(k) = mod(atan2(nv(2),nv(1)),2*pi);
    w(k) = mod(atan2(dot(cross(nv,ev),hv)/H, dot(nv,ev)),2*pi);
end
end
```